



CYART

inquiry@cyart.io

www.cyart.io

AI Context Project

Phase 1 – Week 3 Technical Documentation

Prepared by

AI Context Squad

Date:

17 August 2026



Table of Contents

1. Introduction
2. Objective
3. Technology Stack
4. Memory Management Overview
5. Memory Data Structure
6. Memory API
7. Memory Types and Expiry
8. User Memory Isolation
9. Prompt Template Versioning
10. Prompt Template Authoring
 - 10.1 Asset Summary Prompt Template
 - 10.2 Exposure Explanation Prompt Template
 - 10.3 Remediation Plan Prompt Template
11. Testing and Verification
12. Issues Encountered and Resolutions
13. Conclusion



1. Introduction

The AI Context project under the AEV Platform aims to improve the retrieval and understanding of security asset information using AI. The initial development focused on backend infrastructure and building the first RAG pipeline.

In Week 3, the focus shifts to memory management, including memory creation, retrieval, updating, deletion, user isolation, and removal of outdated information. Prompt versioning is also introduced to organize, update, and reuse prompts.

Structured prompt templates will support three tasks: asset summaries, exposure explanations, and remediation plans. This document presents the implementation details, testing results, code references, and supporting screenshots for the Week 3 development activities.

2. Objective

Week 3 is all about enhancing the AI Context service with a system for basic memory management. The goal of this task is to create the capabilities of the service to be able to store the context that is necessary for the user while managing this information effectively. This week the following objectives should be accomplished: Establish Create, Read, Update, and Delete operations of the memory. The system has to make sure that memory belongs to a user and does not allow the users to access each other's memory.

There has to be a mechanism for disposing the memories that are inactive for over 90 days without use. Create a description of how the stored memory is used while making this information confidential. This task requires the use of versioning of prompt templates so that if some template is changed, previous versions won't get lost. Design three templates for summaries of resources, explanation of exposure, and planning of remediation.



3. Technology Stack

The third week's activities are performed in the project framework where the previous AI Context project was accomplished. The various technologies employed in the project are being used for the backend API, database software, server environment, and tools for the memory and prompt-template component development.

Technology Purpose

Python Backend creation

FastAPI REST API development

PostgreSQL Database

pgvector Vector storage and similarity

SQLAlchemy Interaction with databases

Sentence Transformers Embedding generation

Docker Container of applications

VS Code Environment for programming and development

The memory function is performed by the existing backend, which means that the memory API and prompt template become part of the same AI Context service that has been developed during the previous week.



4. Memory Management Overview

The memory component is used to keep useful pieces of user-specific context that may be relevant to later interactions. Instead of treating every request as completely separate, the service can maintain selected information and make it available when it is needed.

The main idea is to keep memory simple and controlled. A memory item belongs to a particular user, has a type, contains the information that needs to be remembered, and has timestamps that can be used to understand when it was created or last used.

The overall memory workflow can be represented as:

User Request → Memory Check → Memory Operation → Stored User Context → AI Context

The memory layer is not intended to replace the existing asset-search or RAG workflow. Instead, it provides an additional source of context that can be managed separately and used when appropriate.

The Week 3 memory system is divided into a few basic areas:

Memory API – handles the basic create, read, update, and delete operations.

Memory Types – identifies what kind of information is being stored.

User Isolation – makes sure memory remains connected to the correct user.

Expiry Job – removes memories that have remained unused for the defined period.

Privacy Handling – documents how the stored memory is used and protected.



5. Memory Data Structure

The memory data structure defines the information that needs to be stored for each memory item. The exact field names can be confirmed after the final implementation is completed, but the record needs to provide enough information to identify the user, identify the memory category, store the actual memory content, and maintain the relevant timestamps.

The screenshot shows a SQL query editor interface. At the top, there's a toolbar with 'Untitled', 'Save', 'Primary', 'Idle', 'neondb', and some icons. Below the toolbar, a SQL query is entered:

```
1 SELECT column_name
2 FROM information_schema.columns
3 WHERE table_name = 'prompt_templates'
4 ORDER BY ordinal_position;
```

Below the query editor, there's a status bar indicating 'Connected (1 query)'. Below that, there are buttons for 'Run', 'Explain', and 'Analyze'. To the right of these buttons, it shows '538ms' and '6 rows'. Below this, a table of results is displayed:

#	column_name
1	id
2	template_name
3	version
4	template_content
5	is_active
6	created_at

At the bottom right of the table, there are icons for copy, download, and print.



6. Memory API

The Memory API gives us the ability to perform CRUD operations on memories per user.

The list of implemented operations is as follows:

- Create – Creates a new memory.
- Read – Retrieves the memory of a user.
- Update – Updates the memory.
- Delete – Deletes the memory.

The user ID is used in memory operations through the API.

AEV AI Context Service 4.0.0 OAS 3.1	
Swagger UI	
AI Context Service with Memory CRUD, User Isolation, Memory Types, Template Versioning and Asset Vector Search	
default ^	
GET	/ Home
GET	/health Health
GET	/memory Get All Memory
POST	/memory Create Memory
GET	/memory/user/{user_id} Get User Memory
PUT	/memory/{memory_id} Update Memory
DELETE	/memory/{memory_id} Delete Memory
POST	/copilot/asset-search Asset Search
POST	/templates Create Template
PUT	/templates/{template_name} Update Template
GET	/templates/{template_name}/versions Get Template Versions



7. Memory Types and Expiry

The memory system supports four memory types:

Memory Type	Purpose
Preference	Stores user preferences
Workflow	Stores recurring workflow information
Role Context	Stores role-related context
Recent Topic	Stores recently discussed topics

The expiry process can be represented as:

Stored Memory → Check Last Use → Compare with Expiry Period → Remove Eligible Memory

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

#12 resolving provenance for metadata file
#12 DONE 0.0s
[+] up 3/3
✓ Image aev_ai_context-ai-context Built 3.0s
✓ Network aev_ai_context_default Created 0.1s
✓ Container aev-ai-context Started 0.9s
PS C:\Users\krtan\OneDrive\Desktop\AEV_AI_CONTEXT> docker compose exec ai-context python -m app.expiry_job
Expiry cutoff: 2026-05-19 14:08:05.783334+00:00
Expired memories found: 0
DRY RUN: No memories were deleted.
PS C:\Users\krtan\OneDrive\Desktop\AEV_AI_CONTEXT> docker compose exec ai-context python -m app.expiry_job
Expiry cutoff: 2026-05-19 14:09:55.140107+00:00
Expired memories found: 1
Memory ID: 7 | User: user2 | Last Used: 2026-05-18 14:09:34.395210
DRY RUN: No memories were deleted.
PS C:\Users\krtan\OneDrive\Desktop\AEV_AI_CONTEXT>
```




8. User Memory Isolation

User memory isolation prevents memory that belongs to one user from being accessed, updated, and deleted by another user.

Memory functions first validate the user ID and then execute the operation. In doing so, the contextual information that is stored stays linked to the right user.

The expected behaviour is:

1. User A can create and access User A's memory.
2. User A cannot read User B's memory.
3. User A cannot update User B's memory.
4. User A cannot delete User B's memory.

```
Problems Output Debug Console Terminal Ports
powerShell + - [ ] [ ] ... | [ ] [ ] X

../usr/local/lib/python3.11/site-packages/fastapi/testclient.py:1
/usr/local/lib/python3.11/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; instal
l `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

app/schemas.py:29
/app/app/schemas.py:29: PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be
removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
  class MemoryResponse(BaseModel):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 1 passed, 2 warnings in 34.56s =====
PS C:\Users\krtan\OneDrive\Desktop\AEV_AI_CONTEXT>
```



9. Prompt Template Versioning

Prompt templates are used to give the AI model a clear set of instructions for a particular task. As the project develops, these instructions may need to be changed or improved.

The basic versioning flow is:

Template v1 → Template Updated → Template v2 → Current Version

For example:

Template	Version	Status
Asset Summary	1	Inactive
Asset Summary	2	Active

Untitled Save Primary Active neondb Refresh Fullscreen

```
1 SELECT
2   template_name,
3   version,
4   is_active,
5   created_at
6 FROM prompt_templates
7 ORDER BY template_name, version;
```

Connected (1 query)

Run Explain Analyze 336ms 5 rows

#	template_name	version	is_active	created_at
1	asset_summary	1	f	2026-08-17 14:19:26.451942
2	asset_summary	2	t	2026-08-17 14:20:34.431058
3	asset_summary_template	1	t	2026-08-17 14:22:57.001748
4	exposure_explanation	1	t	2026-08-17 14:23:21.120824
5	remediation_plan	1	t	2026-08-17 14:24:13.919966

Copy Download Print



10. Prompt Template Authoring

Three structured prompt templates were created for the AI Context service:

10.1 Asset Summary Prompt Template

Asset Summary Prompt Template is used for creating an effective summary of the asset based on the information about that asset. The prompt template takes into consideration a number of vital elements such as asset type, operating system, IP address, risk score, and security context.

Prompt Template:

“Summarize the asset including asset type, operating system, IP address, risk score, and vital security context.”

10.2 Exposure Explanation Prompt Template

The purpose of the Exposure Explanation Prompt Template is to give an explanation for the security exposure of an asset. This template makes use of the available information regarding the asset to give an explanation of security exposures.

Prompt Template:

“Explain the security exposure of the asset based on its configuration, operating system, public accessibility, and risk score.”



10.3 Remediation Plan Prompt Template

The Remediation Plan Prompt Template was developed in order to facilitate the development of practical recommendations for remediation of identified security risks. The template is geared toward the prioritization of measures to be undertaken depending on the risk to the asset and security recommendations.

Prompt Template:

"Generate a practical remediation plan for the asset. Prioritize measures and make security recommendations."

The screenshot shows a database interface with a dark theme. At the top, there's a header bar with 'Untitled', a 'Save' button, a 'Primary' status indicator, an 'Active' status indicator, a 'neondb' database selector, and some utility icons. Below the header is a SQL editor with a query. The query is a SELECT statement from a table named 'prompt_templates', filtering for 'template_name' in ('asset_summary_template', 'exposure_explanation', 'remediation_plan') and ordering by 'template_name'. Below the editor, there's a status bar indicating 'Connected (1 query)' and buttons for 'Run', 'Explain', and 'Analyze'. To the right of these buttons, it shows '438ms' and '3 rows'. At the bottom, there's a table with 3 columns: '#', 'template_name', 'version', and 'is_active'. The table contains 3 rows of data.

```
3  version,
4  is_active
5  FROM prompt_templates
6  WHERE template_name IN (
7    'asset_summary_template',
8    'exposure_explanation',
9    'remediation_plan'
10 )
11 ORDER BY template_name;
```

Connected (1 query)

Run Explain Analyze 438ms 3 rows

#	template_name	version	is_active
1	asset_summary_template	1	t
2	exposure_explanation	1	t
3	remediation_plan	1	t



11. Testing and verification

Tests were carried out to validate some of the main features for Week 3.

The following features have been validated:

- Memory creation.
- Memory retrieval for a specific user.
- Memory updating.
- Memory deletion.
- Memory types.
- User isolation.
- Expiry of memory after 90 days (dry run).
- Prompt template versioning.
- Three prompt templates.

The automated user-isolation test completed successfully, and the expiry dry run correctly identified an eligible expired memory without deleting it.



12. Issues encountered and resolutions

The following problems occurred during testing and have been addressed:

Issue	Resolution
Httpx dependency missing during testing	Added httpx to project dependencies and rebuilt the Docker container
Modulenotfounderror: No module named "app"	Corrected the Python path while running pytest inside Docker
Docker service	Started the service using Docker Compose
Test path issue inside container	Corrected the pytest file path

After addressing these issues, the isolation test ran automatically successfully.



13. Conclusion

Week 3 focuses on extending the AI Context service with two important areas: user-specific memory management and a structured prompt-template system. The memory component provides the basic CRUD operations required to manage stored context, while the isolation rules help keep that context associated with the correct user.

The memory system also introduces four categories—preference, workflow, role context, and recent topic—and includes an expiry process for memories that have not been used for 90 days. These features provide a more organized way of handling contextual information instead of allowing old or unrelated information to remain indefinitely.

The prompt-template work adds versioning and three templates for asset summaries, exposure explanations, and remediation plans. Keeping these prompts separate and versioned makes it easier to maintain them as the AI Context service develops.

The work completed in Week 3 provides the next layer on top of the RAG foundation developed earlier. After the implementation and testing are finalized, the memory and prompt-template components can be further integrated and refined as part of the broader AEV Platform.